

Fundamentos de DevOps e Versionamento de Código



Conteudista: Prof. Esp. Jônatas Carvalho

Revisão Textual: Esp. Anna Carolina Guimarães

Objetivo da Unidade:

- Compreender os conceitos fundamentais de DevOps e os conceitos de versionamento de código, utilizando as tecnologias Git e GitHub.

  Material Teórico

  Referências



Material Teórico

Terminologias DevOps

DevOps é um jeito de pensar e um modo de trabalho no qual as equipes compartilham histórias e experiências de tal forma que pessoas pratiquem seus ofícios de maneiras mais eficazes. Diversos profissionais pensam que DevOps são ferramentas específicas, como Github, Docker ou Jenkins, mas esse pensamento é errado. DevOps não está delimitado por quais ferramentas são usadas, e sim porque as ferramentas são usadas. Como afirmam Debois *et al.* (2016, p. 57), “o verdadeiro valor de DevOps está em criar um ambiente onde as equipes possam colaborar de maneira eficaz, quebrando silos e fortalecendo a empatia.”

DevOps não é só mais uma metodologia de desenvolvimento de software, porém é relacionada e influenciada por outras metodologias como Agile ou XP, e sua prática pode incluir métodos de desenvolvimento de software ou características como entrega contínua, automação de infraestrutura. Esses conceitos estão presentes e podem ser vistos com frequência em ambientes de DevOps, porém focar somente neles faz com que DevOps perca seu verdadeiro poder, que são os aspectos culturais e interpessoais.

Se tivéssemos que definir DevOps em uma frase, essa seria: DevOps é uma abordagem de desenvolvimento de software que mescla processos, tecnologias e pessoas para uma entrega contínua de valor para os clientes.

Principais Práticas e Ferramentas de DevOps

Temos diversas práticas e ferramentas que são ligadas a DevOps que são desenhadas para integração e automatização dos processos de desenvolvimento, como: testes, entregas e desenvolvimento de software. Tanto as práticas quanto as ferramentas ajudam a ganhar agilidade, eficiência e qualidade em todo o ciclo de vida de um software.

Algumas práticas que podemos citar são:

- **Integração Contínua (CI):** é a prática de integração frequente do código de diferentes desenvolvedores em um repositório público ou disponível dentro da empresa. A integração contínua é importante, pois há a possibilidade de detecção e correção de erros rapidamente, reduzindo os conflitos de código que provavelmente existirão quando mais de um desenvolvedor trabalha no projeto;
- **Entrega Contínua (CD):** de certa forma é uma extensão da integração contínua, que nada mais é que a entrega de códigos que foram previamente validados para ambientes de testes ou produção, e essa ferramenta é muito importante, pois garantirá que o código sempre esteja pronto para a implantação;
- **Monitoramento contínuo:** é a prática de monitorar aplicações, infraestrutura e logs continuamente para a identificação de problemas rapidamente, o uso dessa prática é de vital importância, pois, através do monitoramento contínuo, conseguimos identificar gargalos, indisponibilidade de aplicações e desempenho de todo um ecossistema, isso pode trazer resultados notórios para os clientes que utilizam a aplicação;
- **Automação de teste:** essa prática consiste na criação de *pipelines* automatizadas para execução de tipos diferentes de testes, como unitário, integração, regressão entre outros. É importante termos testes automatizados, pois isso pode aumentar a qualidade do código, reduzir a falha e, também, acelerar as entregas;
- **Cultura de colaboração:** essa prática consiste no encorajamento que equipes de desenvolvimento, infraestrutura e qualidade trabalhem juntas, pois isso traz

grande ganho na melhoria de comunicação e entregas mais eficientes.

Perceba que todas as práticas citadas, nenhuma envolve a correlação com alguma ferramenta. Como já citado anteriormente, DevOps não se trata de ferramentas e sim de um movimento na mesclagem de tecnologias e pessoas.

Abaixo, você encontrará uma tabela com as principais ferramentas utilizadas em DevOps, e deve ser ressaltado que existem uma infinidade de ferramentas, mas foi optado por mostrar apenas as mais relevantes que são utilizadas no mercado.

Tabela 1 – Principais ferramentas de DevOps

Categoria	Ferramenta	Descrição
Gestão de Código e Repositórios	Git	Sistema de controle de versão distribuído.
	Github	
	GitLab	Plataforma de hospedagem de repositórios.
	Bitbucket	

CI/CD (Integração Contínua/Entrega Contínua)	Jenkins	Servidor de automação <i>open-source</i> .
	GitHub Action	Plataforma de CI/CD integrada ao GitHub.
	GitLab CI/CD	Plataforma de CI/CD integrada ao Gitlab.
	Circle CI	Plataformas de CI/CD em nuvem.
	Travis CI	
	Azure DevOps	
Containerização e Orquestração de <i>containers</i>	Docker	Plataforma de containerização.
	Kubernetes	Plataforma de orquestração de <i>containers</i> .
	OpenShift	
Infraestrutura como Código (IaC)	Terraform	Ferramenta de provisionamento de infraestrutura.

Monitoramento	Prometheus	Sistema de monitoramento e alertas.
	Grafana	Sistema de visualização de dados (comumente utilizado junto ao Prometheus).
	New Relic	Sistema de monitoramento e alertas.
	Splunk	Plataformas de monitoramento e análise de dados.
	Datadog	
Cloud	AWS	Plataformas de nuvem.
	Microsoft Azure	
	Google Cloud	

Ferramentas de Segurança DevOps	SonarQube	Plataforma de análise de código estático.
	Snyk	Ferramenta de gerenciamento de vulnerabilidades.

Fonte: Acervo do conteudista

Embora as ferramentas sejam grandes facilitadoras, o core do DevOps é focado na abordagem cultural, medindo seu sucesso não pelas ferramentas implementadas, mas sim pelo impacto na forma em que as equipes trabalham e entregam seus resultados.

Principais Métricas de Desempenho

Quando se trabalha em uma empresa, nós como colaboradores sempre seremos cobrados por resultados, com isso, sempre é importante medirmos a qualidade e a eficiência em nossos processos, principalmente em todo o ciclo de vida de um software, desde sua fase embrionária até a entrega e o uso do cliente. Em DevOps, vamos nos deparar com a palavra *DORA* (*DevOps Research and Assessment*) que são as quatro métricas fundamentais:

- *Deployment Frequency* (Frequência de Implantação);
- *Lead Time for Changes* (Tempo de Ciclo de Implantação);
- *Change Failure Rate* (Taxa de Falha de Mudanças);

- *Mean Time to Restore* (Tempo Médio para Restaurar o Serviço).

Livro

Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations

O livro explora as métricas DORA e é baseado em um estudo em como as equipes de tecnologia alcançam sucesso.

FORSGREN, N. *The science of Lean Software and DevOps: building and scaling high performing technology organizations*. Portland: IT Revolution, 2018.

Essas quatro métricas são essenciais para a avaliação da maturidade e do desempenho das entregas das equipes. Através dessa métricas, as organizações podem ter um grande embasamento para alinhar as operações e os objetivos estratégicos, promovendo, assim, uma cultura de melhoria contínua que irá beneficiar equipes envolvidas, clientes e o próprio negócio.

Versionamento de Código – Introdução ao Git e GitHub

Quando o assunto é Git, a comunidade de desenvolvedores entende que é o básico que um desenvolvedor precisa saber. Independente da área de atuação, em algum momento da vida, o desenvolvedor vai se deparar com alguma situação em que deverá utilizar versionamento de código.

A documentação oficial do Git o denomina como um sistema de controle de versão distribuído de código aberto e gratuito, criado para lidar com qualquer tipo de projeto, sejam eles pequenos ou grandes. Ok, mas o que significa controle de versão distribuído? Com o uso do Git, é possível manter um histórico das alterações dos arquivos de um projeto, sabendo por quem e quando um arquivo foi editado, podendo ainda voltar para estados antigos do projeto.

O Git foi criado, pois a equipe que estava desenvolvendo o kernel do Linux precisava de um sistema que controlasse as versões sem que o código de um desenvolvedor sobrescrevesse o código do outro. O Git para permitir o modo colaborativo utiliza *branches* (ramificações), no qual cada *branch* é uma linha de tempo em que são realizados os *commits* (marcos). Dessa forma, os arquivos podem ser alterados livremente sem impactar outras ramificações.

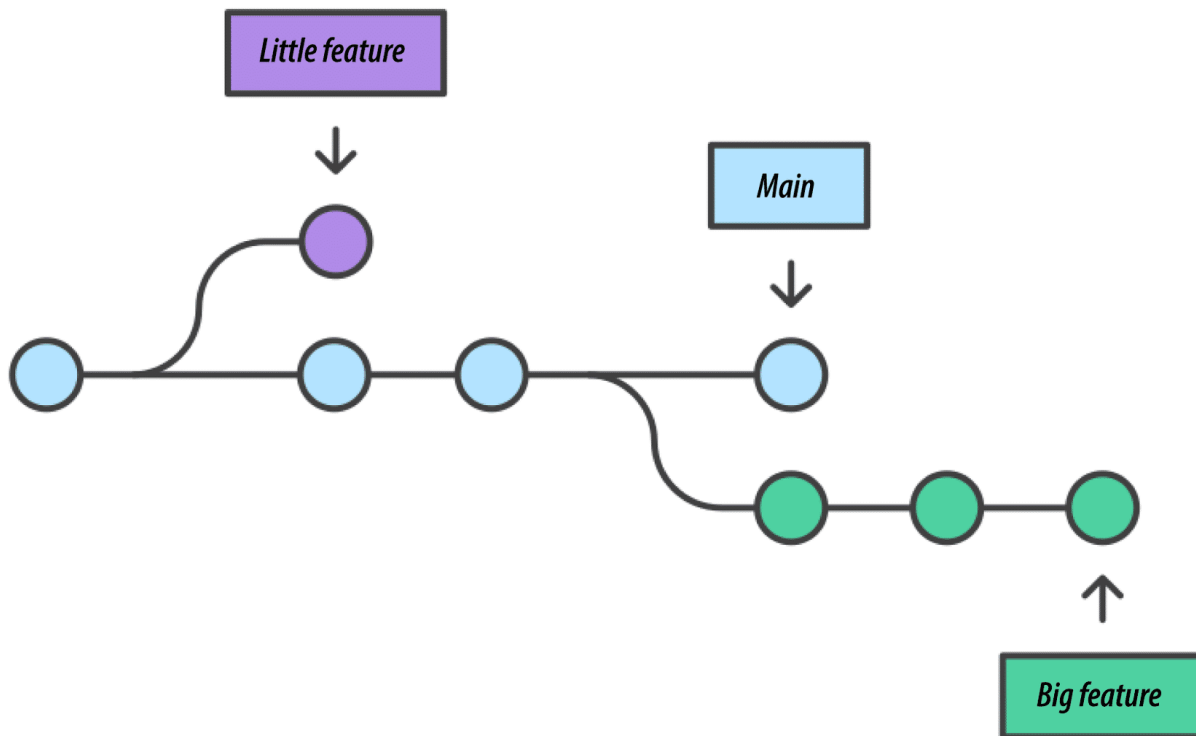


Figura 1 – Branches Git

Fonte: Reprodução

#ParaTodosVerem: a imagem ilustra *branches* com *commits*. Na imagem, temos 3 *branches* denominadas: *Little Feature*, *Main* e *Big Feature*, a *branch Main* possui o primeiro *commit* e entre o primeiro e o segundo nasce a *branch Little Feature*, entre o terceiro e o quarto *commit* da *branch Main* nasce a *branch Big Feature*. Fim da descrição.

Normalmente, o nome Git sempre será atrelado com GitHub ou algum outro sistema de hospedagem de arquivos e códigos-fonte, como: GitLab, Bitbucket, SourceForge entre outros. Como já citado, o GitHub é um sistema de hospedagem de códigos-fonte, atualmente ele é o sistema mais utilizado no mundo e sua popularidade se dá por ser open source, fácil integração com o ecossistema de desenvolvimento como AWS, Google Cloud, ferramentas de CI/CD e IDEs como Visual Studio Code. Além disso, segundo um artigo publicado no site Showmetech, o GitHub possui uma forte comunidade global com cerca de 100 milhões de desenvolvedores registrados e uma

imensa adoção empresarial, a chance de no mercado de trabalho utilizarmos a dupla Git e GitHub é quase que certa.

Sempre que o assunto for versionamento de códigos devemos ter claramente a ideia de que o Git faz toda a organização das *branches* e *commits*, deixando nosso código versionado de maneira correta e sem ser sobrescrito por outro desenvolvedor, e o Github armazena nosso código possibilitando, também, realizar *reviews* de código, *merges*, entre outras coisas.

Conceitos Básicos Git e GitHub

Antes de partirmos para algo mais prático, gostaria de introduzir alguns conceitos básicos sobre essas tecnologias. São eles: **repositório**, **commit**, **branch**, **merge**, **pull request**, **working directory**, **staging area**.

Glossário

- **Repositório:** um repositório é o local em que o código-fonte e seu histórico de alterações estão armazenados. Normalmente, criamos um repositório no momento em que digitamos o comando `git init` que logo iremos aprender;
- **Working directory (diretório de trabalho):** é o diretório em que trabalhamos localmente, ele contém os arquivos do repositório Git e é nele que editamos, adicionamos ou removemos arquivos manualmente;

- **Staging area:** é uma área em que podemos chamar de intermediária na qual podemos preparar as mudanças que desejamos incluir no próximo *commit*. Também podemos selecionar apenas algumas alterações específicas do *working directory* para serem incluídas no próximo *commit*;
- **Commit:** é como se fosse uma foto do projeto capturada por nós naquele determinado momento, ou seja, ele salva o estado atual dos arquivos que foram anteriormente adicionados à *staging area*. Os *commits* são identificados por um *hash* que é um identificador único facilitando o rastreio de mudanças, identificação de quem fez a mudança, facilitando no processo de reversão de alterações caso elas venham causar problemas em um ambiente produtivo;
- **Branch:** é uma ramificação do repositório que normalmente é usada para criar novas *features* ou trabalhar em correções sem estar afetando o código da *branch* principal que normalmente é chamada de *main* ou *master*. Um exemplo que gosto de usar com meus alunos é que as *branches* são como o conceito de multiverso que é muito utilizado em quadrinhos e filmes de super-heróis no qual as *branches* são universos paralelos que partiram de um mesmo ponto de partida, mas por efeitos singulares mudam ao decorrer do tempo;
- **Merge:** é quando entrelaçamos duas *branches*, ou seja, quando combinamos os códigos de uma *branch* com a outra. Aqui devemos tomar um grande cuidado, pois é aqui que diversos desenvolvedores cometem falhas e, por vezes, ficam até com medo de trabalhar com Git. No *merge*, podem acontecer conflitos, pois dois desenvolvedores modificaram a mesma parte de um arquivo, assim destoando do que temos na *master* e no momento de fazer o *merge* ele dará conflitos e é aí que está o problema, pois a mesclagem das *branches* não será realizada de forma automática, obrigando ao desenvolvedor quais partes devem entrar e quais devem sair do projeto;
- **Pull request:** carinhosamente chamada de PR. trata-se de um pedido para uma possível mesclagem de uma *branch* em outra.

Normalmente esse pedido é revisado por outros desenvolvedores que trabalham no mesmo contexto. Isso é feito a fim de que as alterações não sejam aceitas automaticamente e tenham um processo de curadoria mais aguçado evitando a subida de bugs para ambientes produtivos.

Agora com os conceitos básicos esclarecidos, podemos de fato partir para algo mais prático.

Principais Comandos Git

O possui dezenas de comandos que podem ser utilizados. Separei os principais comandos do Git:

- **git config - global user.name “Nome do Usuário”**: esse comando serve para configurar seu nome no sistema Git, assim todos os seus *commits* ficarão com seu nome;
- **git config -- global user.email “email_do_usuario@email.com”**: esse comando serve como identificação, porém ele também é utilizado para vincular os *commits* ao seu perfil no GitHub;
- **git clone**: esse comando clona/faz o download de um repositório no GitHub para nossa máquina;
- **git init**: inicializa um novo repositório Git no diretório em que o terminal está;
- **git add nome_do_arquivo** ou **git add**: esse comando envia os arquivos para a área de preparação ou *staging area*, a diferença é que o primeiro comando adiciona apenas um arquivo, e o segundo adiciona todos os arquivos do diretório;
- **git commit -m “Mensagem que descreve a alteração”**: esse comando salva as alterações no histórico do repositório;

- **git rm nome_do_arquivo:** esse comando remove os arquivos do repositório, é muito útil quando não queremos adicionar um arquivo ao versionamento;
- **git status:** esse comando exibe o estado atual do repositório, mostrando os arquivos que estão rastreados e não rastreados;
- **git log:** esse comando exibe o histórico de *commits*;
- **git diff:** esse comando deve exibir a diferença entre arquivos ou *commits*;
- **git branch** ou **git branch nova_branch:** o primeiro comando exibe o nome da *branch* atual em que estamos, já o segundo cria uma nova *branch*, outra forma de criar uma nova *branch* é através do comando **git checkout -b nova_branch**;
- **git checkout:** esse comando muda de uma *branch* para outra;
- **git merge:** esse comando combina duas *branches*;
- **git reset --hard HEAD~1:** esse comando desfaz o último *commit* e suas alterações;
- **git remote:** esse comando faz o gerenciamento de repositórios remotos, por exemplo, quando criamos nosso repositório local e queremos associá-lo a um repositório no GitHub utilizamos o comando **git remote add origin:**
<https://github.com/nome_do_usuario/nome_do_repositorio.git>.
- **git fetch origin:** esse comando traz as alterações do repositório remoto sem mesclar com nosso repositório local;
- **git pull:** esse comando é como se fosse a combinação de dois comandos, o **git fetch** e o **git merge**, ou seja, o comando **git pull** busca as alterações do repositório remoto e combina com a *branch* local;
- **git push:** envia as alterações locais para o repositório na web.

Dentre todos os comandos do Git, esses citados acima já dão uma boa base para trabalharmos bem no dia a dia, porém garanto que, na maioria dos casos, você vai utilizar os comandos **git init** e **git clone** para criar e inicializar o repositório. Para realizar alterações **git add**, **git commit** e **git status**, e para realizar as sincronizações **git pull** e **git push**. E, para trabalhar com as *branches*, irá utilizar **git branch**, **git checkout** e **git merge**.

Os outros comandos são utilizados em apenas casos específicos ou em casos em que as *branches* apontam duplicidade de código, ou seja, conflitos.

Como Configurar um Repositório Remoto

Vamos agora criar um exemplo real criando um miniprojeto, a ideia do projeto é simples: criar um conversor de números decimais para números binários e expor a aplicação em um *endpoint*.

A aplicação vai ser criada em Node.js, por isso, devemos ter instalado o node em nosso computador. A instalação é bem simples, basta acessar o link, fazer o download e executar.

Site

Node.js

Clique no botão para conferir o conteúdo.

ACESSE

Após a instalação, crie um diretório e dentro deste diretório crie um arquivo chamado `index.js`, dentro do arquivo vamos inserir o seguinte código:

```
const express = require("express");

const app = express();
const port = 3000;

// Endpoint para converter decimal para binário
app.get("/to-binary/:decimal", (req, res) => {
  // Converte o parâmetro para número inteiro
  const decimal = parseInt(req.params.decimal, 10);

  // Se não for número decimal retorna um erro com status 400
  if (isNaN(decimal)) {
    return res.status(400).json({ error: "Invalid decimal number" });
  }

  const binary = decimal.toString(2); // Converte o número decimal
  para binário res.json({ decimal, binary }); // Retorna a resposta como JSON
});

app.listen(port, () => {
  console.log(`Servidor rodando na porta ${port}`);
});
```

Após criar o arquivo e copiar o código, vamos inicializar o projeto com o comando `npm init -y`. Se estiver utilizando o Visual Studio Code, basta abrir um terminal e digitar o comando. Após isso, vamos realizar a instalação do Express.js, para isso, vamos digitar o comando `npm install express`.

Feito isso, já temos nossa aplicação criada; para rodarmos, basta apenas digitar o código `node index.js` e, então, o terminal exibirá a mensagem *Server running on http://localhost:3000*.

Agora, vamos fazer uma chamada para nosso endpoint criado em nossa API. Para isso, basta abrir qualquer navegador e digitar a URL `http://localhost:3000/to-binary/numero_que_desejamos_converter`, substituindo `numero_que_desejamos_converter` por um número decimal válido, para fins de teste vou digitar o número 10, com isso, a URL `http://localhost:3000/to-binary/10`, como retorno vamos receber um json exibindo o número que informamos e esse mesmo número convertido para binário. O retorno esperado deve ser similar à imagem seguinte:

```
// http://localhost:3000/to-binary/10

{
  "decimal": 10,
  "binary": "1010"
}
```

Figura 2 – Retorno do conversor de decimal para binário

Fonte: Acervo do conteudista

#ParaTodosVerem: imagem exibe um json que é o retorno da chamada da nossa aplicação no qual foi passado o número decimal 10 e recebeu como retorno o número binário 1010. Fim da descrição.

Agora vamos fazer uma sequência de comandos em nosso projeto que vai criar o repositório, adicionar os arquivos na *staging area*, commitar as mudanças e, depois, subir o repositório para o nosso repositório no GitHub.

- Para iniciar o repositório Git:
 - `git init;`

- Para adicionar os arquivos na *staging area*:
 - `git add;`
- Mensagem que desejamos deixar registrada no *commit*:
 - `git commit -m "adicionando index.js";`
- Mudando o nome da *branch* de *master* para *main*:
 - `git branch -M main;`
- Associando o repositório no GitHub com nosso repositório local:
 - `git remote add origin`
`git@github.com:nome__do__usuario/nome__do__repositório.git`
- Subindo as alterações locais para o GitHub:
 - `git push -u origin main.`

Feito isso, finalizamos toda a questão da criação e subida dos códigos para o repositório no GitHub. Porém, vamos elevar nossa experiência. Vamos supor que outro desenvolvedor vai realizar uma nova *feature* adicionando um *endpoint* com a funcionalidade de conversão de número decimal para número hexadecimal. Supondo que o outro desenvolvedor já possui o repositório em sua máquina, ele deve executar o comando **git checkout -b feature-hexadecimal** para criar a nova *branch*. Feito isso, ele deve adicionar o código ao final do código anterior:

```
// Novo endpoint para converter decimal para hexadecimal
app.get("/to-hex/:decimal", (req, res) => {
  const decimal = parseInt(req.params.decimal, 10);
  if (isNaN(decimal)) {
    return res.status(400).json({ error: "Invalid decimal number" });
  }
  // Converte para hexadecimal e deixa em maiúsculas
```

```
const hex = decimal.toString(16).toUpperCase();
res.json({ decimal, hex });
});
app.listen(port, () => {
  console.log(`Server running on http://localhost:${port}`);
});
```

Agora, devemos executar os seguintes comandos:

- Para adicionar o arquivo que foi modificado para a *staging* area:
 - `git add index.js;`
- Mensagem que desejamos deixar registrada no *commit*:
 - `git commit -m "Add conversion from decimal to hexadecimal";`
- Subindo as alterações locais para o GitHub:
 - `git push origin feature-hexadecimal.`

Supondo que todas as aprovações foram realizadas com sucesso, agora vamos mergear nossa *feature* na *branch main*, antes vale uma explicação, para podermos mergear uma *branch* na outra, devemos fazer o *checkout* para a *branch* que vai receber as alterações e só então fazer o merge da *feature* que queremos mergear, como segue na imagem abaixo:

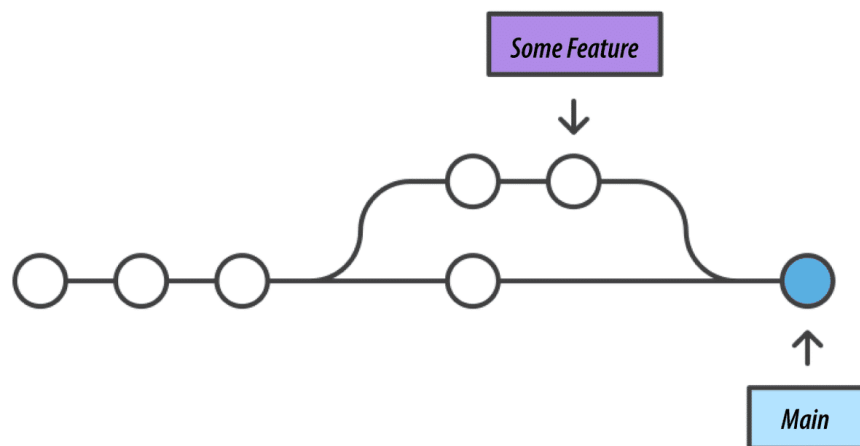
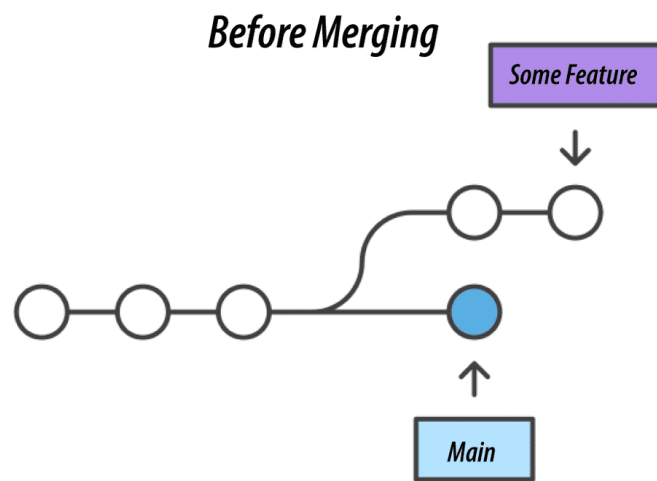


Figura 3 – Margeando *feature* na *branch main*

Fonte: Reprodução

#ParaTodosVerem: a imagem ilustra o *merge* de uma *branch* chamada de *Some Feature*, que é criada entre o terceiro e quarto *commit* e, após dois *commits*, o usuário volta para a *branch master* realizando, assim, o *merge* da *feature*. Fim da Descrição.

Então, para isso, vamos fazer o checkout para a *branch main* e, então, mergear a *branch feature-hexadecimal* com os seguintes comandos:

- Para fazer o *checkout* para a *branch main*:
 - `git checkout main;`
- Para mergear a *branch feature-hexadecimal* na *branch main*:
 - `git merge feature-hexadecimal.`

Feito isso, chegamos ao fim de todo o ciclo da criação de uma nova *feature*, desde a criação do repositório até a adição dos códigos e *merge* para a *branch* principal do projeto.



Referências

ATLASSIAN. *Using branches*. Disponível em:

<<https://www.atlassian.com/br/git/tutorials/using-branches>>. Acesso em: 28/12/2024. 2024.

DEBOIS, J.; KERSTEN, N.; LLOYD, D.; MILLER, J. *Effective DevOps: building a culture of collaboration, affinity, and tooling at scale*. 1. ed. Sebastopol: O'Reilly Media, 2016.

GIT. **Documentação oficial**. Disponível em: <<https://git-scm.com/doc>>. Acesso em: 28/12/2024.

GITHUB. **Documentação oficial**. Disponível em: <<https://docs.github.com>>. Acesso em: 28/12/2024.

GOMES, L. GitHub chega a 100 milhões de desenvolvedores.

Showmetech, 07/01/2024. Disponível em:

<<https://www.showmetech.com.br/github-chega-a-100-milhoes-desenvolvedores/>>. Acesso em: 28/12/2024.